

Multiplayer AI — Problem Statement, Scope & Feature Spec

2026-09-19 · @Someone

Problem Statement

AI agents are now the most powerful tool a team has, but they're still used like a personal notepad. One person opens a chat, types a prompt, and gets an answer in a box only they can see. When a teammate needs to get involved, the best available option is a link to a read-only transcript they can watch but not touch.

That single-player model breaks down as agents start running longer, higher-stakes work — tasks that take hours or days and naturally pull in several people: an incident that needs an on-call engineer and a lead, a deal that needs a rep and a sales engineer, a contract that needs two lawyers, a model that needs an analyst and a reviewer. Today that work gets trapped in one person's private thread. Handoffs happen through screenshots, pasted transcripts, or someone re-explaining the whole task from scratch to the next person. There is no shared surface where a team can watch an agent work, redirect it together, and hand it off the way they would with any other teammate.

Description & Vision

A shared, live workspace where a team and an AI agent work on the same task at the same time — not a transcript to read, but a session anyone on the team can drop into, watch unfold, redirect, or take over, the way Figma turned solo design files into something a team edits together. In the RFS's own framing: what Google Docs did to Word and Figma did to Photoshop, this does to AI agent sessions.

The horizontal version of this — one platform for every function — is already being fought over by well-capitalized players (Dust, Elvex, Taskade, Microsoft) racing to own governance, connectors, and cross-functional reach. The wedge here is the opposite bet: pick one vertical the RFS itself names — **engineers coding together with a shared live agent** — and build the actual purpose-built collaborative session for it, rather than a generic chat wrapper with shared context. Two or more engineers and one coding agent share the same live session: the same file view, the same terminal output, the same plan, with real-time presence showing who's driving and who's watching, and clean handoff

when someone leaves and another picks up mid-task. Prove the real-time multiplayer mechanic works for one deep, technical workflow before expanding to the RFS’s other named verticals — sales, support, legal, analysts, marketing.

Scope

v1 target customer: engineering teams of roughly 5-50 engineers who already use an agentic coding tool (Claude Code, Cursor, or similar) and want to run a session with more than one human driving it at once.

	In scope (v1)	Out of scope (v1, deferred)
Vertical	Engineering / coding sessions only	Sales, support, legal, analysts, marketing
Participants	2-5 concurrent humans + 1 agent per session	Larger sessions, multiple agents per session
Session hosting	Cloud-hosted shared session state	Local-machine-bound sessions (Clairvoyance's model)
Client	VS Code extension + web dashboard	Other IDEs (JetBrains, Vim, etc.)
Cross-org	Single organization only	Cross-company sessions (e.g., vendor + customer)
Deployment	Hosted SaaS	On-prem / air-gapped
Compliance	None required to start	SOC 2, HIPAA, and similar certifications

The out-of-scope items are the natural expansion path once the multiplayer mechanic is proven on one vertical — see the roadmap below.

Target End-Users

Company-level fit is the engineering teams defined in Scope; within that team, several distinct people actually touch the product day to day.

Persona	Role	Why they'd use it	Typical trigger
Pair/mob programmers	Mid-to-senior engineers who already pair or mob	Bring an AI agent in as a third set of hands on a task, without losing the ability to see and steer each other's work	Tackling an unfamiliar part of the codebase or a hard refactor together
On-call / incident responders	The on-call engineer plus whoever they escalate to	The escalated teammate joins with full context the agent already gathered, instead of starting the investigation from zero	A production incident needing a second set of eyes fast
Tech leads / staff engineers	Senior ICs who review or steward agent-driven work	Drop into a live session to redirect the agent's approach before wasted work piles up, rather than reviewing a finished PR after the fact	Reviewing a junior engineer's or an agent's in-progress approach on a significant change
New hires	Engineers ramping on an unfamiliar codebase	Pair with a senior engineer and an agent at once; the senior engineer jumps in exactly when needed instead of the new hire being stuck alone with just the agent	Onboarding, roughly weeks 1-4

Persona	Role	Why they'd use it	Typical trigger
Remote/distributed engineers	Teams spread across time zones or offices	Co-drive one agent session without a live screen-share call — a lighter substitute for real-time pairing over video	Any task two remote engineers would otherwise pair on over a call
Engineering managers	EM or team lead checking in	Peek into a session's progress and leave a quick redirect without derailing the session or scheduling a sync	Checking on a high-priority task's progress

Buyer vs. user: the day-to-day end-users above are individual contributors; the economic buyer is typically an engineering manager, Head of Platform, or VP Engineering evaluating it for team velocity and onboarding time, while the technical champion driving adoption is usually a staff or senior engineer already frustrated by today's screenshot-and-transcript handoffs.

End-to-End Feature Pipeline

```
flowchart LR
  A[Session Creation & Invite] --> B[Shared Live Canvas]
  B --> C[Multiplayer Control]
  C --> D[Handoff]
  D --> E[Audit & History]
  E --> F[Notifications & Async Catch-up]
  F --> G[Integration Layer]
```

- 1. **Session Creation & Invite** — a participant starts a live session from the IDE or CLI, invites teammates via a link, and the coding agent joins as a participant alongside them.
- 2. **Shared Live Canvas** — every participant sees the same file view, terminal output, and running plan/todo list update in real time, synced the way Figma or Google Docs sync edits.

3. **Multiplayer Control** — any participant can type instructions to the agent, pause or redirect it, or take manual control of one file while the agent keeps working elsewhere; presence indicators show who's driving and who's watching.
4. **Handoff** — a participant can leave and another can pick up mid-task with the full context of what the agent has done and why, without anyone re-explaining the task from scratch.
5. **Audit & History** — a version history distinguishing which changes came from which human versus the agent, replayable like a document's edit history.
6. **Notifications & Async Catch-up** — teammates not currently in the session get a live-updating summary and can jump in the moment something needs their input.
7. **Integration Layer** — connects to the team's existing repo, ticketing (Linear/Jira), and chat (Slack) so a session's outcome flows back into the tools the team already works in.

Competitive Landscape

This is the most contested of the ideas researched for this project — several players are already shipping real product against this exact RFS, one at meaningful scale.

Company	Approach	Key features shipped	Gap vs. this plan
Dust.tt	Enterprise shared-agent platform	\$40M Series B, 3,000+ customers (Datadog, 1Password, Vanta), 70+ connectors, 20+ models, dual-layer permissions, audit logs	Horizontal and connector-based, not a real-time simultaneous multiplayer session; governance-first, not workflow-depth-first
Clairvoyance	Shipped live multiplayer agent sessions	Multiple people in one live session, local-first (host machine), cross-platform, iPhone app, persistent memory	Host-machine-bound, not cloud-native or enterprise-hardened; no vertical-specific workflow tooling
Taskade Genesis	Real-time human-agent co-editing	Operational-Transform sync (same tech as Google Docs), 4 collaboration modes (Handoff, Parallel, Hierarchical, Peer Review), shared cursors/memory/triggers	Targets no-code ops teams (sales ops, support ops, content ops), not engineering/coding workflows
Elvex	Enterprise shared "Spaces"	Individual/Team/Temporary Spaces, SOC 2 + HIPAA, spans Product/IT/HR/Sales/Legal/Healthcare, 20+ integrations	Persistent shared context, not real-time simultaneous multiplayer editing of one live task

Company	Approach	Key features shipped	Gap vs. this plan
Intracscope	Reapplying to YC for this exact RFS	Shared multi-model access, governance, "Manifests" for reusable context; live sessions on roadmap	Live sessions not yet shipped; targets agencies/ops broadly, not one deep vertical
Microsoft 365 / Teams SDK	Platform-level infra for collaborative agents	Session continuity, threaded conversations, human-in-the-loop approvals; GitHub Copilot, Atlassian Rovo, Linear, and Cursor already embed on it	Infrastructure for others to build on, not an out-of-box product; ties adopters to the Teams ecosystem
Miro Canvas 26	Shared AI workspace bolted onto a whiteboard	Shared AI workspaces for teams, agents, and workflow automation	Canvas/brainstorming-oriented, not a live coding or deep single-workflow session

Takeaway: every player here is horizontal (spanning many functions) or infrastructure-level (a platform for others to build on). Only Taskade and Clairvoyance have real simultaneous multiplayer editing, and neither is purpose-built for engineering/coding workflows — that specific, deep vertical is the open gap this plan targets.

Feature Comparison: Our Implementation vs. Competitors

The approach-level picture above holds up at the feature level: real, shipped capability exists across the board, but no one combines live simultaneous editing with a workflow purpose-built for engineering.

Feature	Our v1	Dust.tt	Clairvoyance	Taskade Genesis	Elvex	MS 365 / Teams SDK
Real-time simultaneous multiplayer editing (not turn-based)	✓	—	✓	✓	—	Partial (session continuity, not live cursors)
Live presence (who's driving / who's watching)	✓	—	Partial	✓	—	—

Feature	Our v1	Dust.tt	Clairvoyance	Taskade Genesis	Elvex	MS 365 / Teams SDK
Redirect the agent or take over a file mid-session	✓	—	Partial	Partial (next-turn, not instant)	—	Partial (approval-based, not live redirect)
Clean handoff to a new participant with full context	✓	Partial	Partial	Partial	Partial	Partial
Audit/version history: human vs. agent changes	Roadmap	✓	—	—	Partial	—
Purpose-built for engineering/coding workflows	✓	— (horizontal)	— (generic)	— (targets no-code ops)	— (horizontal)	Partial (partners like Cursor build on it)
Cloud-hosted (not bound to one host machine)	✓	✓	—	✓	✓	✓
Lives inside the IDE, not a separate app/canvas	✓	—	—	—	—	—
Async digest for teammates outside the session	Roadmap	Partial	—	—	—	✓
Enterprise governance (SOC 2 / HIPAA)	— (deferred)	✓	—	—	✓	✓
Multiple specialized agents in one session	Roadmap	Partial	—	✓ (Hierarchical, Peer Review modes)	Partial	Partial
Cross-organization sessions	Roadmap	—	—	—	—	—

What makes this different: the three rows nobody else has covered together — live simultaneous editing, purpose-built for engineering, and lives inside the IDE — are exactly the combination this plan targets. What we deliberately don't have yet: enterprise

governance (Dust and Elvex are well ahead here) and mature audit history, both explicitly deferred rather than ignored — they're the price of entry once this expands past the engineering wedge into regulated or enterprise-scale customers.

Stress-Testing Uniqueness: What About AI-Native IDEs?

The comparison above checks against workflow/agent platforms (Dust, Elvex, etc.), but the sharper question is whether Cursor, Windsurf, Zed, or VS Code — tools that already own the "IDE-native + agentic coding" pieces — have already shipped a live-multiplayer layer that closes this gap. As of September 2026, checked directly:

Player	What they actually have	Multiplayer + agent, together?
Zed	Native real-time multiplayer editing (its original differentiator, pre-dating the AI wave) — shared cursors, live edits between humans	No. Zed's docs treat AI agents and multiplayer collaboration as separate systems; agents run single-player even in a shared project
Cursor	AI agent sessions, static shared transcripts, sequential "Team Follow-ups" on cloud agents	No. Cursor staff confirmed on their own forum (Sept 2026) that real-time shared local-agent sessions are not built and not currently planned — it's an open feature request
Windsurf (Wave 13)	"Parallel Agents" / Arena Mode	Misleading name for this purpose — it's one developer running multiple agent instances side-by-side via git worktrees, not multiple humans in a session
VS Code	Multi-agent orchestration (Feb 2026): run Claude, Codex, and Copilot agents in parallel	Same pattern — one developer managing several agents, no human co-editing

Player	What they actually have	Multiplayer + agent, together?
Omnigent (Databricks, open-source, June 2026)	Cross-device session continuity, shared live-session viewing, "co-attach" where a teammate's messages execute on your machine	Closest real attempt, but explicitly turn-based — only one person drives at a time, not simultaneous multi-user control

Verdict: no AI-native IDE or agent framework has shipped genuine simultaneous multi-human-plus-agent live co-editing as of this research. The specific combination this spec is built around — real-time simultaneous control, IDE-native, purpose-built for engineering workflows — is not yet occupied. But this is not empty white space either: Zed already owns the multiplayer infrastructure and could bolt on agent participation at any time; Databricks is already circling from the agent-orchestration side with Omnigent's turn-based "co-attach"; and Microsoft, Cursor, and Windsurf are all actively shipping in this exact adjacent area every few weeks. The uniqueness here is a **timing-based wedge, not a structural moat** — a first-mover window that could close within one to two product cycles if any of these players decides simultaneous multiplayer is worth building rather than turn-based sharing.

Recommended Technical & Agentic Stack

The core technical challenge here is new relative to a typical agent product: merging concurrent edits from several humans and an agent on the same files, in real time, without conflicts — the same problem Figma and Google Docs solved for human-only editing, now with an agent as an additional writer.

Category	Tools	Role in the pipeline
Real-time sync layer	Yjs or Automerge (CRDT libraries)	The heart of the product: merges concurrent human + agent edits to the same files without conflicts
Transport / presence	WebSocket-based provider (self-hosted, or a managed layer like Liveblocks/PartyKit)	Live cursors, who's-driving/who's-watching presence, low-latency updates

Category	Tools	Role in the pipeline
Agentic core	Claude Agent SDK, using the same read/edit/diff tool pattern as Claude Code	The agent participant itself: reads files, proposes edits, executes commands inside the shared session
Client	VS Code extension (webview reusing the CRDT client) + a browser-based dashboard for participants without the extension installed	Shared Live Canvas, Multiplayer Control
Backend	Node.js/TypeScript (natural fit for the Yjs/CRDT ecosystem)	Hosts sessions, brokers the sync layer, coordinates the agent's turns
Sandboxed execution	Northflank or E2B for running the agent's commands/tests in isolation	Keeps agent-executed code out of participants' machines directly
Data & storage	PostgreSQL for session and audit-trail metadata, S3-compatible storage for session snapshots/replays	Backs the Audit & History and Notifications features
Integrations	GitHub/GitLab API, Slack API, Linear/Jira API	Integration Layer: repo access, async digests, ticket linkage
Auth & hosting	Clerk or Auth0, Vercel (dashboard), Fly.io/AWS/GCP (session backend)	Product infrastructure
Observability	Sentry, OpenTelemetry, session replay logging	Debugging sync conflicts and agent behavior in live sessions

Verified against current practice (2026-09-19): CRDTs (Yjs/Automerger) remain the standard approach for this class of real-time collaborative editing, and Taskade's own public description of its sync layer explicitly confirms the same Operational-Transform lineage as Google Docs — validating this as the right technical foundation rather than a novel bet.

Success Metrics & Validation Plan

- **Pre-application validation:** 8-10 conversations with engineering teams confirming that driving an agent solo, then handing off by screenshot or transcript, is a real recurring pain — before writing the YC application.
- **MVP proof:** two engineers and one agent complete a real coding task together in a single live session, both able to see and steer the agent in real time.
- **Design partners:** 5-10 engineering teams running genuine live sessions within the first 60 days.
- **Session concurrency:** average number of humans actually present per session (a session nobody co-drives isn't validating the core bet).
- **Handoff success rate:** share of sessions where a new participant picks up mid-task successfully, without needing the previous participant to re-explain it live.
- **Time to onboard mid-session:** how long it takes a newly joined participant to understand what's happened so far and start contributing.

Implementation Roadmap

Each phase gates on the metrics above — move on hitting its exit criteria, not on a calendar date.

Phase	Timeline	Goal	Key deliverables	Exit criteria
0. Validate	Weeks 1-2	Confirm the pain is real before writing code	8-10 conversations with engineering teams about solo-agent handoff pain	3+ teams confirm this is a real, recurring frustration
1. MVP / Proof of Concept	Weeks 3-6	Prove the real-time multiplayer mechanic works, narrowly	CRDT sync layer, VS Code extension, 2 humans + 1 agent in one live session, one sample repo	Two people co-drive one agent through a real coding task in real time

Phase	Timeline	Goal	Key deliverables	Exit criteria
2. Design Partner Pilot	Weeks 7-10	Get it running on real team workflows	Session invite/handoff flow, presence indicators, 3-5 design partners onboarded	Design partners running genuine multi-person sessions; first handoff-success data collected
3. YC Application	Weeks 10-12	Package real usage into an application	Traction metrics, design-partner testimonials, live demo	Application submitted backed by real session data, not just a plan
4. Harden & Expand (Beta)	Weeks 12-20	Make it reliable enough for daily use	Audit/version history, async notifications, GitHub/Slack/Linear integrations, up to 5 concurrent participants	Handoff success rate and time-to-onboard tracked and improving across 10+ active teams
5. Public Launch	Weeks 20-26	Open self-serve signup	Self-serve onboarding, billing (Stripe), marketing site, public launch (Product Hunt / Hacker News)	First paying teams acquired without a manual sales conversation
6. Scale & Expand	Months 6-12	Extend into the RFS's other named verticals	Sales, support, and legal-specific session types; multi-agent sessions; cross-org sessions; seed fundraising	A second vertical live with its own purpose-built session type, not a copy-pasted UI

MVP Implementation Plan (Phase 1 detail)

Objective: prove that two humans and one agent can co-drive a single live coding session in real time, with clean handoff, on one real task. A working two-person session is the only success condition — everything else is deferred.

Scope for this build only:

	In scope (MVP)	Deferred to Phase 2+
Participants	Exactly 2 humans + 1 agent	3-5 participants, multiple agents
Client	VS Code extension only	Web dashboard, other IDEs
Session start	Manual CLI/extension command	Invite links, session discovery
Repo	One purpose-built sample repo	Real customer repos
Auth	None (trusted local testing)	Full auth, org accounts
Handoff	Basic — a participant can leave and the other continues	Full context replay for someone joining fresh

Week-by-week:

Week	Focus	Deliverable
1	Sync layer spike	Stand up Yjs (or Automerge) syncing a single file's edits between two local clients in real time
2	Agent as a third writer	Wire the Claude Agent SDK-based agent into the same CRDT document as a participant, not a separate process
3	VS Code extension	Build the extension surface: shared file view, live cursors, presence indicators (who's driving, who's watching)
4	Multiplayer control	Let either human type instructions to the agent or take manual control of a file while the agent works elsewhere
5	Handoff	One participant disconnects mid-task; the other continues without losing state; agent keeps working through the transition
6	Dogfood + iterate	Run the full flow on 2-3 real coding tasks with actual engineers; fix what breaks; get one design partner reaction to the live demo

Definition of done: two engineers, in two different browser/editor windows, co-drive one Claude Agent SDK session against a real repo — both can see the agent's edits and each other's live, either can redirect it, and one can leave mid-task while the other continues without restarting.